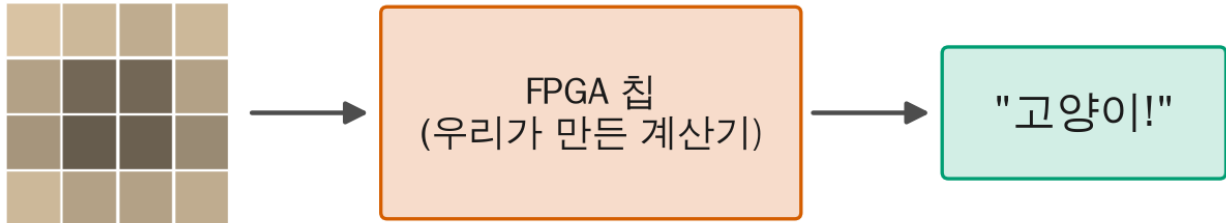


작은 칩에 "그림 알아맞히기 두뇌" 넣기

FPGA CNN 가속기 프로젝트를 초등학생도 알 수 있게 — 그리고 논문은 어디로 가야 하나

사진 (32×32 점)



한 줄 요약. 사진을 보고 "고양이", "자동차"를 알아맞히는 인공지능을 손바닥만 한 칩(FPGA) 안에 직접 만든 계산기로 돌렸다. 사진 128장을 보여줬더니 110장을 맞혔고, 칩이 낸 14,680,064개의 계산 답 가운데 컴퓨터로 확인한 정답과 다른 것은 하나도 없었다.

이 문서의 순서

1. CNN은 그림을 어떻게 보나 2. FPGA는 무엇이고 왜 레고로 만들었나 3. 0 건너뛰기 4. 일꾼을 어떻게 배치할까 5. 칩 하나로 여섯 층 전부 6. 결과 7. 뜻밖의 발견: 입구가 좁다 8. 솔직하게 말할 것 9. 논문은 이렇게 쓰자

1. CNN은 그림을 어떻게 보나

컴퓨터에게 사진은 숫자가 뽀뽀하게 적힌 모눈종이다. CNN(합성곱 신경망)은 작은 돋보기 창을 이 모눈종이 위에서 한 칸씩 옮기며 "여기 뽀족한 귀 모양이 있나?", "여기 바퀴처럼 둥근 게 있나?"를 점수로 매긴다.

돋보기 창 (3×3)

0	3	5	2	0	1
4	7	0	6	3	0
1	0	8	2	5	4
0	6	3	0	2	7
5	2	0	4	1	0
3	0	6	1	0	2

창 안의 숫자 × 정해진 무게(가중치)

→ 전부 더하기 = 특징 점수 1개

창을 한 칸씩 옮기며 그림 전체를 훑는다

곱하기·더하기가 사진 한 장에 3,863만 번!

점수를 매기는 방법은 단순하다. 창 안의 숫자마다 미리 배워 둔 무게(가중치)를 곱하고, 그걸 전부 더한다. 이 "곱하고 더하기"를 층을 바꿔 가며 여섯 번 반복하면 마지막에 "고양이일 점수가 제일 높다"가 나온다.

어려운 건 계산 방법이 아니라 양이다. 사진 한 장에 곱하기·더하기가 38,633,472번 필요하다. 사람이 1초에 한 번씩 하면 1년 3개월이 걸린다.

2. FPGA는 무엇이고, 왜 레고로 만들었나

보통 컴퓨터 칩(CPU)은 공장에서 모양이 정해져 나온다. FPGA는 다르다. 안에 아주 작은 블록이 수만 개 들어 있고, 그걸 어떻게 연결할지를 우리가 정한다. 레고처럼 원하는 기계를 조립하는 칩이다. 우리가 쓴 칩(Zynq-7020)에는 레고 블록(LUT)이 53,200개 있다.

FPGA에는 곱셈만 잘하는 전용 계산기(DSP)도 220개 들어 있다. 그런데 실제 기계에서 FPGA를 쓰는 이유는 보통 따로 있다. 전파 신호를 거르거나, 카메라 영상을 다듬거나, 모터를 제어하는 일이다. 그런 일이 이 전용 계산기를 먼저 차지한다.

칩 안의 전용 계산기 (DSP)



이미 다른 일(필터·영상 처리)을 하는 중

레고 블록 (LUT)으로 조립



작은 블록을 모아 계산기를 직접 만든다

그래서 이 프로젝트는 "전용 계산기는 다른 일에 쓰이고 있다"고 보고, 곱셈기를 전부 레고 블록으로 조립했다. 실제로 CNN을 돌린 설계는 전용 계산기를 0개 썼다.

솔직히 덧붙이면, 우리 실험에서 전용 계산기가 정말 다른 일을 하고 있었던 건 아니다. 레고로만 만들게 한 진짜 이유는 공정한 비교였다. 두 설계 중 한쪽만 전용 계산기를 몰래 쓰면 레고 개수를 비교할 수가 없다. 그리고 보드 빌드에서 0개가 나온 건 "쓰지 마"라고 명령해서가 아니라 도구가 스스로 그렇게 고른 결과다. 논문에도 이 두 가지를 그대로 적었다.

3. 0 건너뛰기

CNN의 중간 층에서는 숫자 중 상당수가 0이다. "여기엔 그 모양이 없다"는 뜻이다. 0에 무엇을 곱해도 0이므로, 우리 계산기는 0을 만나면 곱셈을 하지 않고 넘어간다.

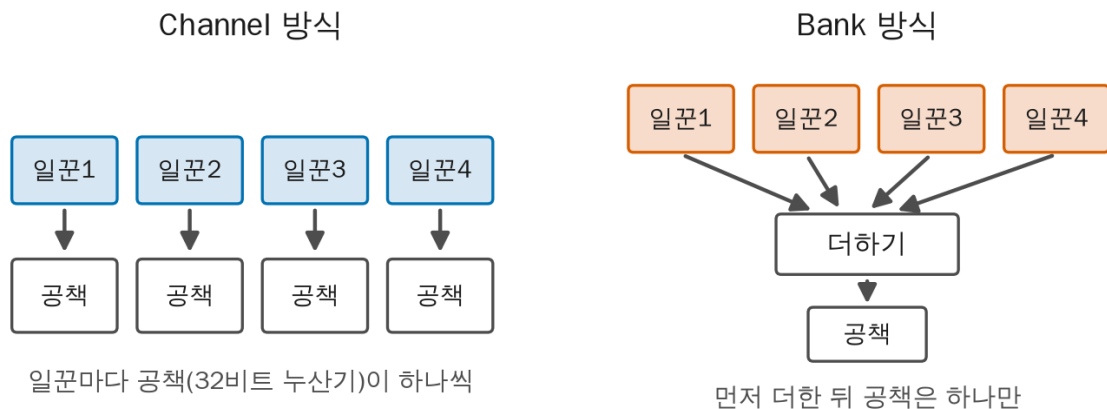


$0 \times \text{무엇} = 0 \rightarrow 0$ 인 칸은 계산을 건너뛴다. 이 칸들 중 절반가량이 0이다.

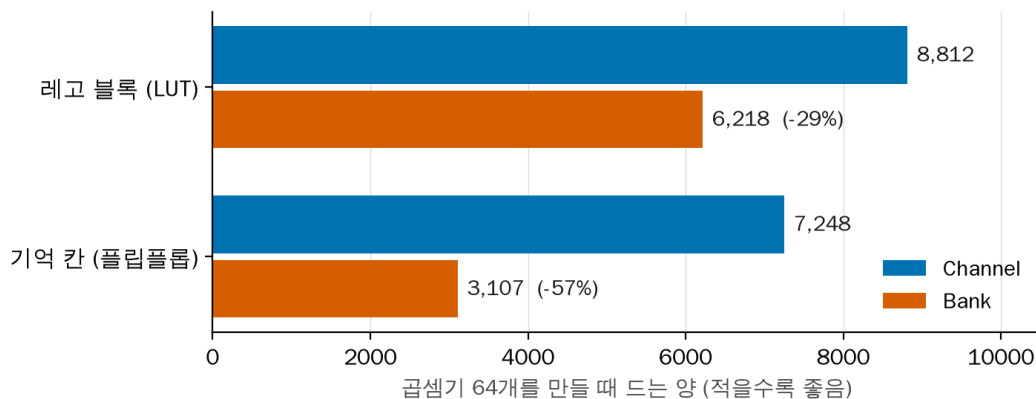
이 네트워크의 마지막 층에서는 숫자의 87.5%가 0이었다(0이 아닌 것이 12.5%). 그만큼 곱셈을 안 해도 된다. 다만 이 절약에는 조건이 붙는다는 것이 7장에서 나온다.

4. 일꾼을 어떻게 배치할까

곱셈기를 일꾼이라고 하자. 우리는 일꾼 64명을 쓸 수 있다. 일꾼을 배치하는 방법이 두 가지 있다.



Channel 방식은 일꾼마다 자기 답을 적을 공책을 하나씩 준다. Bank 방식은 일꾼 여럿이 먼저 답을 더하고, 공책은 하나만 쓴다. 이 "공책"(32비트 누산기)이 생각보다 비싸다.



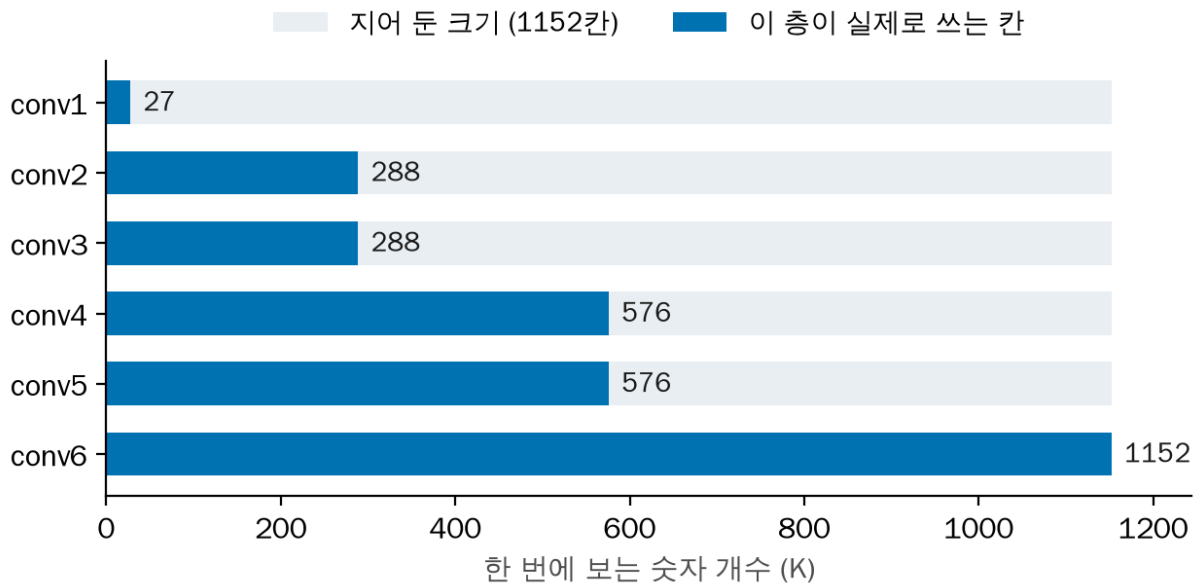
같은 일꾼 64명인데 Bank 방식이 레고를 29%, 기억 칸을 57% 덜 쓴다. 게다가 회로가 단순해서 더 빠른 박자로 돌 수 있어, 윈도우 하나를 끝내는 시간도 $8.96\mu\text{s}$ 대 $8.57\mu\text{s}$ 로 4.4% 짧다. 작으면서 빠르다.

유명한 선행 연구(Ma 등, 2017·2018)는 이 Bank 쪽 방향을 "쓸모없다"며 버렸다. 그 이유는 0을 건너뛰지 않는 경우에만 맞는 이유였다. 0을 건너뛰면 오히려 이쪽이 싸다는 것이 이 연구의 첫 번째 발견이다.

5. 칩 하나로 여섯 층 전부

CNN의 여섯 층은 크기가 제각각이다. 첫 층은 한 번에 숫자 27개를 보고, 마지막 층은 1,152개를 본다. 층마다 칩을 새로 만들면(재합성 40분) 쓸 수가 없다.

그래서 제일 큰 층에 맞춰 방을 크게 지어 두고, 층마다 "이번엔 몇 칸 쓸 거야"를 레지스터 두 개로 알려준다. 칩 하나로 여섯 층을 전부 돌린다.



대신 대가가 있다. 방을 크게 지었으니 칩의 기억 창고(블록 RAM)를 51% 쓴다. 레고는 21%만 쓰는데 창고는 절반이다. "칩 하나로 전부"의 값이 이것이다.

6. 결과

무엇	결과
칩이 낸 계산 답	14,680,064개
그중 정답과 다른 것	0개
사진 알아맞히기	128장 중 110장 (85.9%)
원래(소수점) 인공지능	86.26%
사진 한 장에 걸린 시간	13.24 ms (초당 75.5장)
일꾼 64명이 일한 비율	91%
칩에서 쓴 레고 / 창고 / 전용 계산기	21% / 51% / 0개

여기에 fig9_classified.png(칩이 사진에 붙인 이름표)가 들어간다. 보드를 돌린 컴퓨터에서 python paper/explainer/make_explainer.py를 다시 실행하면 자동으로 들어간다.

"정말 칩에서 돌았나?" 정확도만으로는 증명이 안 된다(칩이 정답과 똑같았으니 칩을 건너뛰어도 같은 정확도가 나온다). 증거는 칩 스스로 센 박자 수다. 이 수는 컴퓨터가 줄 수 없는 값인데, 미리 계산해 둔 예측과 1% 차이로 맞았고, 두 번 돌려도 한 자리도 안 틀리고 같았다.

7. 뜻밖의 발견: 입구가 좁다

일꾼을 늘리면 더 빨라질까? 계산해 보니 아니었다. 우리 계산기는 숫자를 한 박자에 하나씩만 받는다. 갈때기 입구가 좁은 것과 같다.



그래서 한 번에 숫자 288개를 보는 층은 아무리 일꾼이 많아도 최소 288박자가 걸린다. 이게 "입구 한계"다. 칩을 돌려 보니 모든 층이 이 한계에 딱 붙어 있었다.

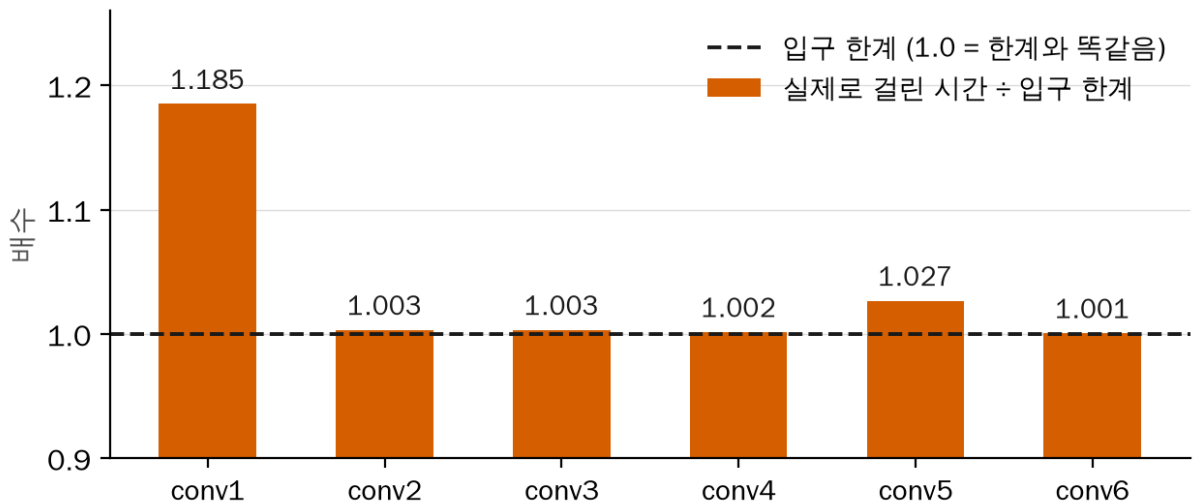


사진 한 장 전체로는 실제 662,051박자, 입구 한계 654,336박자로 1.2%밖에 차이가 안 난다. 그리고 이 한계값은 칩을 돌리기 전에 계산해서 적어 둔 값이다. 예측이 맞았다는 뜻이다. (conv1만 18% 높은 건 창이 27칸으로 너무 작아서, 창 사이 준비 시간 몇 박자가 크게 보이기 때문이다.)

이게 이 연구의 가장 중요한 발견이다. 이 크기의 CNN을 이 크기의 칩에서 돌리면 문제는 일꾼(곱셈기) 수가 아니라 입구다. 일꾼을 두 배로 늘려도 5%만 빨라진다. 더 빨라지고 싶으면 입구를 넓혀야 한다. 4장의 "Channel이나 Bank나"도 이 네트워크에서는 입구 때문에 차이가 거의 안 난다 — 우리 결과의 적용 범위를 우리가 먼저 말해 두는 것이다.

8. 솔직하게 말해야 할 것

논문에서 이걸 숨기면 심사위원이 찾아낸다. 먼저 말하면 신뢰를 얻는다.

한계	설명	논문에서
박자가 느리다	설계는 100 MHz로 돌 수 있게 만들었지만(여유 +0.221 ns), 보드 설정 탓에 50 MHz로 돌아왔다. 시간은 전부 측정한 50 MHz 값이다.	측정값만 주장
칩이 전부 한 건 아니다	곱하기·더하기는 전부 칩. 숫자 재배열, 총 사이 크기 조정, 풀링, 마지막 분류는 PC가 했다.	분담을 명시
CPU와 비교 안 함	공정한 비교 대상은 같은 칩 안의 ARM CPU다. ARM 컴파일러가 없어 아직 못 켜다.	향후 과제로
시험 사진 128장	정확도 표본이 작다. 다만 칩이 정수 모델과 완전히 같으므로 정확도는 모델의 성질이다.	그렇게 설명
네트워크 하나, 칩 하나	CIFAR-10 6층, XC7Z020에서만 측정했다.	범위를 명시

9. 논문은 이렇게 쓰자

이 논문의 주인공은 "Channel이나 Bank나"가 아니라 "DSP 없이 CNN 전체를, 칩 하나로, 틀림 없이 돌렸고, 그 한계가 어디인지 미리 알아맞췄다"다. 축 비교는 "왜 이런 모양으로 만들었나"의 근거로 내려간다.

논문이 말해야 할 세 문장

- ① DSP를 하나도 쓰지 않고, 형상이 다른 여섯 층을 비트스트림 하나로 돌려, 14,680,064개 출력을 오차 없이 재현했다.
- ② 곱셈기 예산이 고정될 때 0을 건너뛰는 가속기에서는 선행 연구가 버린 탭 뱅크 축이 출력 채널 축보다 작고(LUT 29%, FF 57%) 빠르다(4.4%).
- ③ 그러나 이 크기의 네트워크에서는 입력 포트가 한계이며, 실행 전 예측과 1.2% 이내로 맞는다 — 설계 전에 어느 쪽이 한계인지 계산할 수 있다.

절 구성

절	내용	상태
I. 서론	문제(DSP 없음, 네트워크 전체) → 선행 연구의 기각 이유가 희소성에서 무너짐 → 입구 한계 발견을 앞에	영문·한글 완료
II. 구조	작업 단위, 0 건너뛰기, Channel, Bank, 실행 시점 geometry	한글 완료, 영문 보강
III. 방법	CIFAR 양자화, 층별 실행, 등예산 비교, 검증, 합성, 하드웨어	한글 완료, 영문 새로
IV. 결과 1	축 비교(설계 근거로 축소)	완료
V. 결과 2	네트워크 전체: 정확성, 입구 한계 표, 자원, 처리율, fig9	한글 완료, 영문 새로
VI. 논의	결과의 적용 범위(입구 한계), 한계 목록, 다음 할 일	한글 완료, 영문 새로

남은 일, 중요한 순서대로

순서	할 일	왜	비용
1	영문 II~VI를 한글판 구조로 다시 쓰기	투고는 영문	제 작업
2	ARM Cortex-A9 기준선 (SD카드 Linux 추천)	"CPU보다 얼마나?"에 대한 유일한 공정한 답	SD카드 + 1~2시간
3	build_zedboard.tcl에 -max_dsp 0 고정 후 재빌드	DSP 0을 "관측"에서 "보장"으로	40분 + 보드 10분
4	전력 추정 (Vivado 리포트)	이미지당 에너지(J) — FPGA가 이기는 축	빌드 폴더에 있음
5	100 MHz 재측정 (선택)	시간 수치를 설계 능력대로	git 동기화 후 10분
6	급유 경로 넓히기 (다음 논문)	입구를 넓혀야 일꾼이 다시 의미	새 연구

하지 말아야 할 것

· 데스크톱 CPU나 느리게 짤 CPU 코드와 비교해서 "몇 배 빠르다"고 쓰지 않기. 아는 심사위원은 바로 알아본다.

- 100 MHz에서 151 fps를 "측정했다"고 쓰지 않기. 측정은 50 MHz, 75.5 fps다.
- 정확도를 "하드웨어가 돌았다는 증거"로 쓰지 않기. 증거는 박자 수와 스톨 카운터다.
- 85.9% 대 85.7% 차이를 의미 있게 해석하지 않기. 표본(128장 대 512장)이 다를 뿐이다.

투고처 (제안)

국제 학회라면 FPGA 설계 중심의 FPL, FPT(ICFPT) 같은 학회가 이 규모의 하드웨어 측정 논문과 잘 맞는다. 국내라면 대한전자공학회(IEIE) 학술대회 논문지가 한글판을 그대로 쓸 수 있다. 지도교수님과 정할 부분이니 참고만 하면 된다.

모든 수치 출처: `paper/figures/data.py` · 이 문서 생성: `paper/explainer/make_explainer.py`